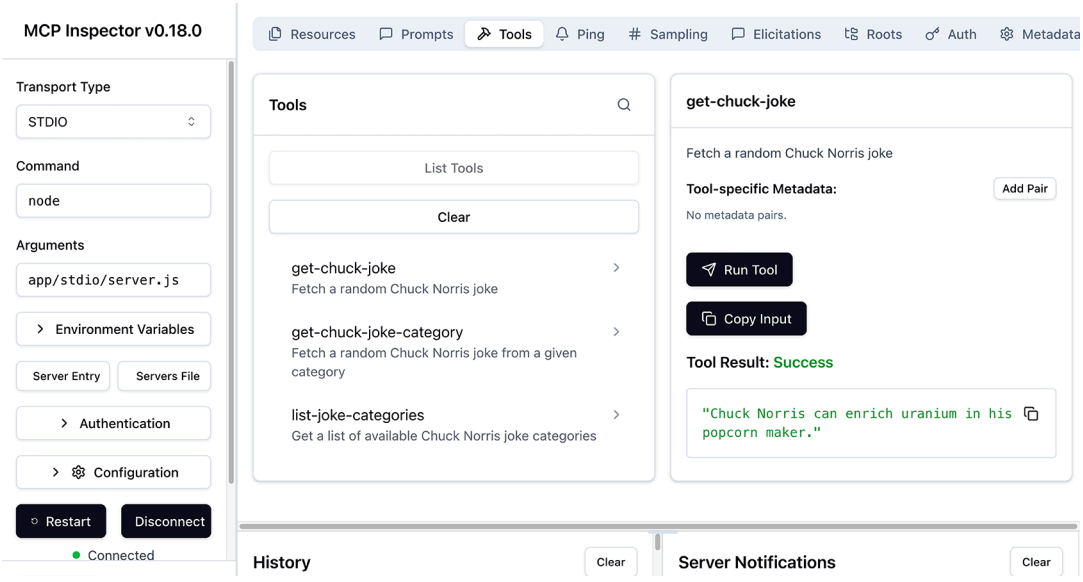


Figure 12.6 Using the MCP Inspector tool to test and debug the server implementation

12.4.2 Additional considerations for MCP servers

When designing and deploying MCP servers, it is useful to reflect on both the current specification of the MCP and its future roadmap. MCP servers function as an additional layer of abstraction designed primarily for web applications that rely on LLMs. They allow clients to explore resources, invoke prompts, and orchestrate tools in a structured way, effectively serving as a bridge between AI models and the underlying data or functionality. Despite this specialized role, MCP servers are still servers in the conventional sense, so standard maintenance, security, and operational practices apply. This includes managing API keys, controlling access, monitoring performance, and ensuring reliable uptime.

The MCP specification emphasizes clear resource registration, structured message formats, and tool integration. Each resource or prompt must have a unique identifier and URI, and its responses should be well structured and predictable, enabling clients to reliably fetch and interact with content.

By keeping both the AI-focused abstraction and the server responsibilities in mind, developers can build MCP servers that are robust, secure, and maintainable while still enabling powerful AI orchestration.

12.5 Integrating MCP servers with LangChain.js

We've integrated the Vercel AI SDK with the MCP to expose external APIs as callable tools. Now let's see how the same MCP server can be integrated into a Next.js application using LangChain.js.